

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering
Jackfruit Mini Project - Software Engineering (UE24CS341A)

Software Requirements Specification
for Project Management System

Field	Details
Project Title	Project Management System
Group Number	Team 14
Version	1.0 approved
Prepared By	Samarth M H, Sameeksha Prasanna Kumar, Sanapureddy Hemanth Kumar Reddy, Sanjana Guttapally
Organization	PES University, Department of Computer Science and Engineering
Date Created	05 September 2026

Table of Contents

1. Introduction	3
1.1 Purpose.....	3
1.2 Intended Audience and Reading Suggestions	3
1.3 Product Scope	3
1.4 References.....	3
2. Overall Description.....	3
2.1 Product Perspective.....	3
2.2 Product Functions	3
2.3 User Classes and Characteristics.....	4
2.4 Operating Environment.....	4
2.5 Design and Implementation Constraints	4
2.6 Assumptions and Dependencies.....	4
3. External Interface Requirements	4
3.1 User Interfaces	4
3.2 Software Interfaces	4
3.3 Communications Interfaces.....	4
4. Analysis Models	5
5. System Features	5
5.1 Authentication, Workspace and Access Control	5
5.2 Project and Task Management	6
5.3 Agile Sprint and Time Management	6
5.4 Real Time Collaboration, Notifications and Analytics	7
6. Other Nonfunctional Requirements.....	7
6.1 Performance Requirements	7
6.2 Safety Requirements	8
6.3 Security Requirements	8
6.4 Software Quality Attributes	8
6.5 Business Rules	8
7. Other Requirements.....	8
8. Validation Specification (Initial Planning).....	8
9. Project Plan	9
10. Tool Stack.....	9
Appendix A: Glossary	10
Appendix B: Field Layouts	10
Appendix C: Requirement Traceability Matrix	10

Revision History

Name	Date	Reason For Changes	Version
Team 14	05 September 2026	Initial version of the Software Requirements Specification	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for Project Management System, a web based application that allows a team to organize projects, manage tasks on a Kanban board, plan Agile sprints, track time and view project progress through a live analytics dashboard. This document covers the complete first version of the system and is intended to be the single reference for what the system must do.

1.2 Intended Audience and Reading Suggestions

This document is intended for the development team building the system, the course instructor evaluating the project, and any reviewer who needs to understand the expected behaviour of the system. Section 2 gives an overall picture of the product. Section 3 covers external interfaces. Section 4 covers the analysis models. Section 5 lists the detailed functional requirements grouped by feature. Section 6 covers non functional, security and business rule requirements. The appendices contain the glossary, field layout and traceability matrix.

1.3 Product Scope

Project Management System is a new, self contained web application. It allows a team to create workspaces and projects, break work into tasks and subtasks, plan work using an Agile backlog and sprints, log time against tasks, collaborate through comments and a live activity feed, and monitor progress through an analytics dashboard. Task updates are pushed to connected users in real time. The system is being built as a course mini project and is scoped for use by a small team rather than a large organisation.

1.4 References

- Jackfruit - Software Engineering Project Guidelines, PES University, 2026
- Software Engineering - Level 3 Mini Project Student Submission Guide, PES University
- Team 14 Synopsis / Project Proposal document

2. Overall Description

2.1 Product Perspective

Project Management System is a new, standalone product and is not a follow on to an existing system. It consists of a React based frontend, a Django REST Framework backend, a PostgreSQL database, and a WebSocket layer built using Django Channels for pushing live updates to connected clients. The frontend communicates with the backend using a REST API for standard operations and a WebSocket connection for real time task and activity updates.

2.2 Product Functions

- User registration, login and role based access control within a workspace
- Creation and management of workspaces and projects
- Task and subtask creation, assignment and Kanban board management
- Task dependency management with cycle detection
- Agile backlog and sprint planning with story point estimation
- Time logging against tasks
- Real time task board updates using WebSockets
- Comments, mentions and a live activity feed
- In app notifications
- Analytics dashboard showing burndown, velocity and workload

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
Owner	Full control of a workspace, including deleting it and assigning any role	Sets up the workspace, typically one per workspace
Admin	Manage projects, members and roles other than Owner	Manages day to day workspace operations
Member	Create and manage tasks assigned to them, log time, comment	Regular contributor, most frequent user of the system
Viewer	Read only access to projects and dashboards	Stakeholder who needs visibility but does not edit content

2.4 Operating Environment

The system is a web application accessed through a modern browser such as Chrome, Firefox or Edge, on desktop or laptop devices. The backend runs on Python 3 with Django REST Framework and Django Channels, deployed on a Linux based server or container. The database is PostgreSQL. The frontend is a React single page application. A Redis instance is used as the channel layer backing for WebSocket connections.

2.5 Design and Implementation Constraints

- The backend must be implemented using Django REST Framework in Python
- Real time updates must be implemented using Django Channels over WebSockets
- The frontend must be implemented using React
- PostgreSQL must be used as the primary database
- The project must follow Agile methodology with sprint based delivery
- Git and GitHub must be used for version control, with CI/CD through Jenkins
- The project must be completed within the timeline defined by the course schedule

2.6 Assumptions and Dependencies

- Users are assumed to have a modern browser and a stable internet connection
- Third party libraries such as React, Django REST Framework and Django Channels remain available and compatible during development
- The system depends on a Redis instance being available for the WebSocket channel layer
- The project assumes a small team of users for demonstration purposes rather than large scale production load

3. External Interface Requirements

3.1 User Interfaces

The system provides a responsive web interface with the following primary screens: login and registration, workspace and project dashboard, Kanban task board, backlog and sprint planning screen, task detail panel with comments and activity feed, analytics dashboard, and a notification panel. Every screen follows a consistent layout with a persistent navigation sidebar and a header showing the current workspace and project.

3.2 Software Interfaces

- Django REST API exposing JSON endpoints consumed by the React frontend
- PostgreSQL database accessed through the Django ORM
- Redis used as the channel layer for Django Channels WebSocket connections

3.3 Communications Interfaces

- HTTPS is used for all standard REST API requests and responses
- WSS (WebSocket Secure) is used for real time task board and notification updates

- All API responses use JSON as the data format

4. Analysis Models

The table below lists the primary use cases supported by the system, along with the primary actor, precondition and expected outcome for each. A corresponding use case diagram and class diagram are maintained separately as part of the design documentation.

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Register and Login	Any User	Application available	Authenticated session created
UC-02	Create Workspace	Owner	User logged in	New workspace created
UC-03	Invite Member	Owner / Admin	Workspace exists	Member added with assigned role
UC-04	Create Project	Member	Workspace exists	New project created
UC-05	Create and Assign Task	Member	Project exists	Task created and assigned
UC-06	Move Task on Board	Member	Task exists	Task status updated in real time
UC-07	Plan Sprint	Member	Backlog has items	Sprint created with assigned items
UC-08	Log Time	Member	Task assigned to user	Time entry recorded
UC-09	Comment and Mention	Member	Task exists	Comment saved, mentioned user notified
UC-10	View Analytics Dashboard	Owner / Admin / Member	Project has data	Dashboard displayed with current metrics

5. System Features

5.1 Authentication, Workspace and Access Control

5.1.1 Description and Priority

Handles user registration, login, workspace creation, member invitations and role based access control.
Priority: High. Owned by Samarth M H.

5.1.2 Stimulus/Response Sequences

- User submits registration details, system creates account and confirms success
- User submits login credentials, system validates and returns an authentication token
- Owner submits an invite request, system generates a unique invite link
- Owner or Admin changes a member role, system updates permissions immediately

5.1.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-AUTH-01	The system shall allow a new user to register with a unique email address and password, and allow a registered user to log in and receive an authentication token.	High	Functional test
FR-AUTH-02	The system shall allow an authenticated user to create a new workspace or organization.	High	Functional test

ID	Requirement	Priority	Verification
FR-AUTH-03	The system shall allow a workspace Owner to invite members using a unique invite link.	High	Functional test
FR-AUTH-04	The system shall allow an Owner or Admin to assign the role Owner, Admin, Member or Viewer to a member, and shall restrict workspace and project actions based on the assigned role.	High	Authorization test

5.2 Project and Task Management

5.2.1 Description and Priority

Handles creation and management of projects, tasks, subtasks and the Kanban board. Priority: High. Owned by Sameeksha Prasanna Kumar.

5.2.2 Stimulus/Response Sequences

- Member creates a project, system adds it to the workspace
- Member creates a task with details, system adds it to the board in the default status column
- User drags a task card to another column, system updates its status
- Member defines a dependency between two tasks, system validates it before saving

5.2.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-TASK-01	The system shall allow an authenticated member to create a project within a workspace.	High	Functional test
FR-TASK-02	The system shall allow a project member to create a task, break it into subtasks, and assign it to one project member.	High	Functional test
FR-TASK-03	The system shall display tasks on a Kanban board organized by status column, and allow a user to move a task between columns using drag and drop.	High	Functional test
FR-TASK-04	The system shall allow a dependency to be defined between two tasks and shall detect and reject a dependency that would create a cycle.	Medium	Negative test

5.3 Agile Sprint and Time Management

5.3.1 Description and Priority

Handles the product backlog, sprint planning, story point estimation and time logging. Priority: High. Owned by Sanjana Guttapally.

5.3.2 Stimulus/Response Sequences

- Member adds an item to the backlog, system stores it as unassigned to any sprint
- Member creates a sprint and moves backlog items into it, system associates the items with the sprint
- Member logs time against a task, system stores the entry against that task and user
- System recalculates the burndown and velocity charts whenever a task status or estimate changes

5.3.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-SPR-01	The system shall allow a project member to add an item to the product backlog and assign it a story point estimate.	High	Functional test
FR-SPR-02	The system shall allow a project member to create a sprint with a defined start and end date and move backlog items into it.	High	Functional test
FR-SPR-03	The system shall allow a user to log time spent against a task assigned to them.	High	Functional test

ID	Requirement	Priority	Verification
FR-SPR-04	The system shall generate a burndown chart showing remaining story points across an active sprint and a velocity chart showing story points completed per sprint.	Medium	Functional test

5.4 Real Time Collaboration, Notifications and Analytics

5.4.1 Description and Priority

Handles live task board updates, comments, mentions, the activity feed, in app notifications and the analytics dashboard. Priority: High. Owned by Sanapureddy Hemanth Kumar Reddy.

5.4.2 Stimulus/Response Sequences

- A user updates a task, system pushes the change to all connected members of that project over WebSocket
- A user posts a comment mentioning a teammate, system saves the comment and notifies the mentioned user
- A user opens the analytics dashboard, system computes and displays current workload and progress metrics

5.4.3 Functional Requirements

ID	Requirement	Priority	Verification
FR-RTC-01	The system shall push a task update to all connected project members in real time using a WebSocket connection.	High	Integration test
FR-RTC-02	The system shall allow a user to add a comment to a task and mention another member within a comment.	High	Functional test
FR-RTC-03	The system shall record task and comment activity in a project activity feed and generate an in app notification when a user is mentioned or assigned a task.	High	Functional test
FR-RTC-04	The system shall display an analytics dashboard showing workload distribution and overdue task counts across project members.	Medium	Functional test

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	95 percent of task board requests shall receive a response within 2 seconds under normal load.	Performance test
NFR-002	Availability	The application shall be available at least 99 percent during the defined operating window, excluding planned maintenance.	Operational review
NFR-003	Usability	A new user shall be able to create and assign a task in no more than 4 primary screens.	Usability walkthrough
NFR-004	Reliability	A WebSocket disconnection shall not cause loss of task data already saved to the database.	Failure/reconnect test
NFR-005	Maintainability	Business logic shall be separated into identifiable modules with automated unit tests.	Code review
NFR-006	Scalability	The system shall support at least 50 concurrent WebSocket connections per project without noticeable delay in updates.	Load test

6.2 Safety Requirements

- The system shall require explicit confirmation before permanently deleting a workspace or project
- The system shall prevent permanent loss of task data by soft deleting tasks before final removal

6.3 Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	Passwords shall not be stored in plaintext.	Code and configuration review
SEC-002	Role based access shall prevent a Viewer from creating or editing tasks.	Authorization test
SEC-003	The system shall validate and sanitize user supplied input before processing.	Negative and security test
SEC-004	Authenticated API and WebSocket connections shall reject expired or invalid tokens.	Security test
SEC-005	A member removed from a workspace shall immediately lose access to that workspace's data.	Authorization test
SEC-006	Security relevant events such as failed login attempts shall be logged.	System test

6.4 Software Quality Attributes

- Maintainability: modules for auth, tasks, sprints and collaboration shall be independently testable
- Portability: the backend shall run on any Linux based environment supporting Python 3 and PostgreSQL
- Reusability: the REST API shall be documented so it can be reused by a future mobile client
- Robustness: the system shall handle invalid input without crashing, returning a clear error response instead

6.5 Business Rules

- Only a workspace Owner can delete a workspace
- A task cannot be marked Done while it has an incomplete dependency
- Only the assigned member or a project Owner can log time against a task
- A sprint cannot be started if it has no backlog items assigned to it
- Only an Owner or Admin can invite or remove a workspace member
- A Viewer has read only access and cannot create, edit or delete any content

7. Other Requirements

- The database shall retain task and activity history for the duration of the project for reporting purposes
- The system shall be deployable using Docker containers for both the backend and frontend
- The system is intended for a single institution deployment and does not require multi tenant data isolation beyond workspace level separation

8. Validation Specification (Initial Planning)

At this stage the validation specification is planning focused. It explains how each category of requirement will eventually be validated once implementation is complete. This is not the final validation or test report, which will be produced after system testing.

Requirement Category	Planned Validation	Example
Functional	Black box functional testing of each feature	Creating a task and moving it across the Kanban board

Requirement Category	Planned Validation	Example
Security	Authentication and authorization testing, basic penetration testing	A Viewer attempting to access an admin only endpoint
Performance	Load and response time testing	Concurrent task board requests from multiple users
Usability	Task based walkthrough with a new user	A new user creating and assigning a task
Reliability	Failure and retry scenario testing	A WebSocket disconnecting and reconnecting mid update
Unit	Automated unit tests with coverage reporting	Story point total calculation for a sprint

9. Project Plan

The project follows an Agile lifecycle delivered across six sprints. The table below breaks the work down by package, with owner and estimated effort in person days.

Work Package	Tasks	Owner	Estimated Effort (Person Days)	Output
WP1	Requirements gathering and use case definition	All	4	SRS
WP2	Architecture and UML design	Samarth / Sameeksha	4	Architecture and design document
WP3	Auth, workspace and access control module	Samarth	7	Working feature
WP4	Project and task management module	Sameeksha	7	Working feature
WP5	Agile sprint and time management module	Sanjana	7	Working feature
WP6	Real time collaboration, notifications and analytics module	Hemanth	8	Working feature
WP7	Testing, CI/CD and security validation	All	6	Test and validation evidence
WP8	Final validation, documentation and report	All	4	Final report

Total estimated effort across the team is approximately 48 person days, distributed across the six sprint schedule described in the project synopsis.

10. Tool Stack

Activity	Tool / Technology	Evidence to Submit
Version control	Git and GitHub	Repository URL, commit history
Agile planning	GitHub Projects / Jira	Backlog, user stories, sprint board
CI/CD	Jenkins	Pipeline screenshot or log
Containerization	Docker	Dockerfile and build/run evidence
Testing	Pytest, Jest	Test results
Static analysis	SonarQube or similar	Analysis report
Real time layer	Django Channels with Redis	WebSocket connection logs
Design	Draw.io	UML diagrams

Appendix A: Glossary

Term	Definition
Workspace	A top level container that groups projects and members, owned by a single Owner
Project	A collection of tasks within a workspace, worked on by assigned members
Task	A unit of work within a project that can be assigned, tracked and moved across board columns
Subtask	A smaller unit of work that belongs to a parent task
Backlog	A list of work items that have not yet been assigned to a sprint
Sprint	A fixed time period during which a defined set of backlog items are worked on
Story Point	A relative unit used to estimate the effort required to complete a backlog item
Burndown Chart	A chart showing remaining story points in a sprint over time
Velocity	The average number of story points a team completes per sprint
RBAC	Role Based Access Control, restricting actions based on assigned user role
WebSocket	A persistent connection protocol used to push real time updates from server to client
Kanban Board	A visual board that displays tasks organized into status columns

Appendix B: Field Layouts

The table below lists the primary fields used across the core entities of the system.

Field	Length	Data Type	Description	Mandatory
user_id	36	UUID	Unique identifier for a user	Y
email	100	String	Unique login email address	Y
password_hash	128	String	Hashed user password	Y
workspace_id	36	UUID	Unique identifier for a workspace	Y
role	20	Enum	Owner, Admin, Member or Viewer	Y
project_id	36	UUID	Unique identifier for a project	Y
task_id	36	UUID	Unique identifier for a task	Y
task_title	150	String	Title of the task	Y
status	20	Enum	Backlog, To Do, In Progress, Done	Y
story_points	5	Integer	Estimated effort for a backlog item	N
sprint_id	36	UUID	Unique identifier for a sprint	N
time_logged_minutes	10	Integer	Time logged against a task in minutes	N
comment_text	1000	String	Content of a comment on a task	Y

Appendix C: Requirement Traceability Matrix

At this stage of the project, architecture, design and code references are marked as TBD and will be completed once the corresponding design and implementation artifacts are produced.

Req ID	Architecture Ref	Design Ref	Code/File Ref	Test Case ID	System Test ID
FR-AUTH-01	TBD	TBD	TBD	TBD	TBD
FR-AUTH-02	TBD	TBD	TBD	TBD	TBD
FR-AUTH-03	TBD	TBD	TBD	TBD	TBD
FR-AUTH-04	TBD	TBD	TBD	TBD	TBD
FR-TASK-01	TBD	TBD	TBD	TBD	TBD
FR-TASK-02	TBD	TBD	TBD	TBD	TBD
FR-TASK-03	TBD	TBD	TBD	TBD	TBD
FR-TASK-04	TBD	TBD	TBD	TBD	TBD
FR-SPR-01	TBD	TBD	TBD	TBD	TBD
FR-SPR-02	TBD	TBD	TBD	TBD	TBD
FR-SPR-03	TBD	TBD	TBD	TBD	TBD
FR-SPR-04	TBD	TBD	TBD	TBD	TBD
FR-RTC-01	TBD	TBD	TBD	TBD	TBD
FR-RTC-02	TBD	TBD	TBD	TBD	TBD
FR-RTC-03	TBD	TBD	TBD	TBD	TBD
FR-RTC-04	TBD	TBD	TBD	TBD	TBD
SEC-001	TBD	TBD	TBD	TBD	TBD
SEC-002	TBD	TBD	TBD	TBD	TBD
SEC-003	TBD	TBD	TBD	TBD	TBD
SEC-004	TBD	TBD	TBD	TBD	TBD
SEC-005	TBD	TBD	TBD	TBD	TBD
SEC-006	TBD	TBD	TBD	TBD	TBD

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering
Jackfruit Mini Project - Software Engineering (UE24CS341A)

Software Architecture & Design Specification

for Project Management System

Field	Details
Project Title	Project Management System
Group Number	Team 14
Version	1.0 (initial)
Prepared By	Samarth M H, Sameeksha Prasanna Kumar, Sanapureddy Hemanth Kumar Reddy, Sanjana Guttapally
Organization	PES University, Department of Computer Science and Engineering
Date Created	26 September 2026
Related Document	Software Requirements Specification (SRS) for Project Management System, v1.0

Table of Contents

Table of Contents.....	2
1. Introduction	3
1.1 Purpose.....	3
1.2 Scope	3
1.3 References	3
2. Architecture	3
2.1 Component Diagram and Description	3
2.2 Architectural Pattern	5
2.3 Traceability to Requirements	5
2.4 Security Architecture	6
2.5 Other Architectural Considerations.....	7
2.5.1 Deployment View.....	7
2.5.2 Extensibility.....	7
3. Design	8
3.1 UML Sequence Diagrams	8
3.2 API Design	9
3.3 Error Handling.....	10
3.4 Other Design Considerations	11
3.4.1 Data Model Notes	11
3.4.2 Consistency of Real-Time Updates	11

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design for the Project Management System described in the Software Requirements Specification (SRS), version 1.0. It records the architectural pattern chosen for the system, the major components and how they interact, the security architecture, and the detailed design of representative workflows, the API surface and error handling. This document is the single reference used to guide implementation and to evaluate whether the delivered code matches the intended design.

1.2 Scope

The scope of this document covers the backend (Django REST Framework and Django Channels), the frontend (React single page application), the data store (PostgreSQL) and the real-time channel layer (Redis), exactly as scoped in Section 2 of the SRS. It does not repeat the functional requirements themselves, which remain owned by the SRS; instead it explains how those requirements are realised architecturally.

1.3 References

- Software Requirements Specification (SRS) for Project Management System, Team 14, v1.0, 05 September 2026
- Jackfruit - Software Engineering Project Guidelines, PES University, 2026
- IEEE Std 1016-2009, IEEE Recommended Practice for Software Design Descriptions (format reference)

2. Architecture

2.1 Component Diagram and Description

The diagram below shows the system decomposed into a React frontend, four backend domain modules running inside a single Django REST Framework service, a Django Channels WebSocket layer, PostgreSQL and Redis. The four backend modules correspond directly to the four feature groups in SRS Section 5 (5.1-5.4) and are owned by the same four team members who own those requirements.

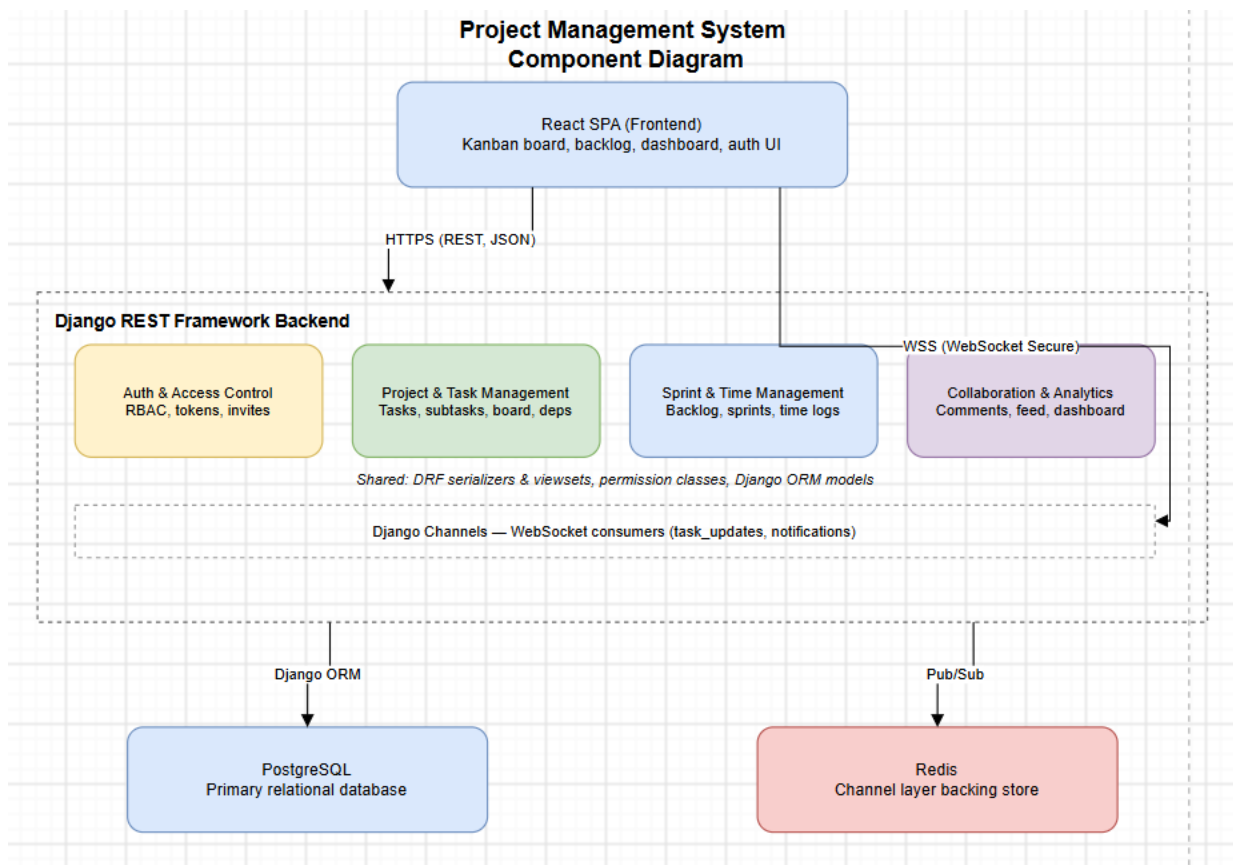


Figure 1: Component diagram of the Project Management System

Component responsibilities:

- **React SPA (Frontend):** renders the login/registration, workspace and project dashboard, Kanban board, backlog/sprint planning screen, task detail panel and analytics dashboard (SRS 3.1); holds one REST client and one WebSocket client.
- **Auth & Access Control:** registration, login/token issuance, workspace invites and RBAC enforcement (FR-AUTH-01 to FR-AUTH-04).
- **Project & Task Management:** projects, tasks, subtasks, the Kanban board and dependency-cycle detection (FR-TASK-01 to FR-TASK-04).
- **Sprint & Time Management:** backlog, sprint planning, story points, time logging and chart data (FR-SPR-01 to FR-SPR-04).
- **Collaboration & Analytics:** comments, mentions, activity feed, notifications and the analytics dashboard (FR-RTC-01 to FR-RTC-04).

- Django Channels: WebSocket consumers that authenticate a connection once and then subscribe it to one project group per open board, backed by Redis as the channel layer.
- PostgreSQL / Redis: system of record for all persisted data, and the pub/sub backing store for real-time fan-out, respectively.

2.2 Architectural Pattern

The system follows a layered client-server architecture with an event-driven extension for real time features:

- Presentation layer: the React SPA, structured as components and hooks with a unidirectional (Flux-style) data flow; it never talks to PostgreSQL or Redis directly.
- Application/service layer: Django REST Framework viewsets and serializers implement the Model-View-Serializer variant of MVC that Django REST Framework encourages - models carry persistence and validation logic, serializers carry request/response shaping, viewsets carry the request-handling flow.
- Data layer: PostgreSQL, accessed exclusively through the Django ORM, so that no module writes raw SQL and all four backend modules share one consistent data-access style.
- Event/notification layer: synchronous, request-response work (create task, log time, add comment) goes through REST; anything that must reach other already-connected users without a page refresh (task moves, comments, notifications) is additionally published to Django Channels over Redis and pushed to WebSocket clients. This keeps the REST API simple and cacheable while still meeting the real-time requirement (FR-RTC-01).

The backend is deliberately built as a single deployable service organised into four internal modules by domain (a "modular monolith") rather than four separate microservices, since SRS Section 2.6 scopes the system for a small team rather than production-scale, multi-service deployment; this keeps the six-sprint schedule in SRS Section 9 realistic while still keeping the modules independently testable, as required by NFR-005.

2.3 Traceability to Requirements

Architecture Component	SRS Section	Requirement IDs Realised
Auth & Access Control module	5.1	FR-AUTH-01 - FR-AUTH-04, SEC-001, SEC-002, SEC-004, SEC-005
Project & Task Management module	5.2	FR-TASK-01 - FR-TASK-04
Sprint & Time Management module	5.3	FR-SPR-01 - FR-SPR-04
Collaboration & Analytics module	5.4	FR-RTC-02, FR-RTC-03, FR-RTC-04
Django Channels + Redis	5.4, 6.1	FR-RTC-01, NFR-004, NFR-006
React SPA	3.1	NFR-003 (usability, task creation in <=4 screens)

2.4 Security Architecture

Security is enforced at the API boundary rather than in the frontend, since the frontend is not trusted:

- Authentication: every REST request and WebSocket connection carries a bearer token issued at login; DRF authentication classes and the Channels connection middleware both reject an expired or invalid token before any view or consumer code runs (SEC-004).
- Authorization: a DRF permission class reads the caller's role (Owner / Admin / Member / Viewer) for the workspace or project in scope and rejects the request with 403 Forbidden before a Viewer can create, edit or delete content, and before a non-Owner can delete a workspace (SEC-002, Business Rules in SRS 6.5).
- Credential storage: passwords are hashed with Django's PBKDF2 (or Argon2) password hasher and never stored or logged in plaintext (SEC-001).
- Input handling: serializers validate and constrain every field (type, length, enum values from SRS Appendix B) before it reaches the ORM, and the ORM's parameterised queries prevent SQL injection (SEC-003).
- Session revocation: removing a member from a workspace immediately invalidates that member's cached permissions for the workspace so the next request is rejected (SEC-005).

- Auditability: failed login attempts and permission-denied events are written to a security log for later review (SEC-006).
- Transport: all REST traffic uses HTTPS and all WebSocket traffic uses WSS, so tokens and task data are never sent in the clear (SRS 3.3).

2.5 Other Architectural Considerations

2.5.1 Deployment View

The backend, frontend, PostgreSQL and Redis are each packaged as a separate Docker container and started together (e.g. via docker-compose in development, and the same images behind Jenkins-built artifacts in CI), matching the containerisation requirement in SRS Section 7. Django Channels workers can be scaled to more than one process behind Redis without any application code change, which is how the 50-concurrent-connection target in NFR-006 is met.

2.5.2 Extensibility

Because the REST API is the only supported way to read or write system state (the frontend uses no private endpoints), SRS Section 6.4's requirement that the API be reusable by a future mobile client is satisfied without further architectural change.

3. Design

3.1 UML Sequence Diagrams

Two representative workflows are detailed below: creating and assigning a task (a synchronous, REST-only flow with a validation branch), and moving a task on the board (a flow that also fans out a real-time update to other connected clients).

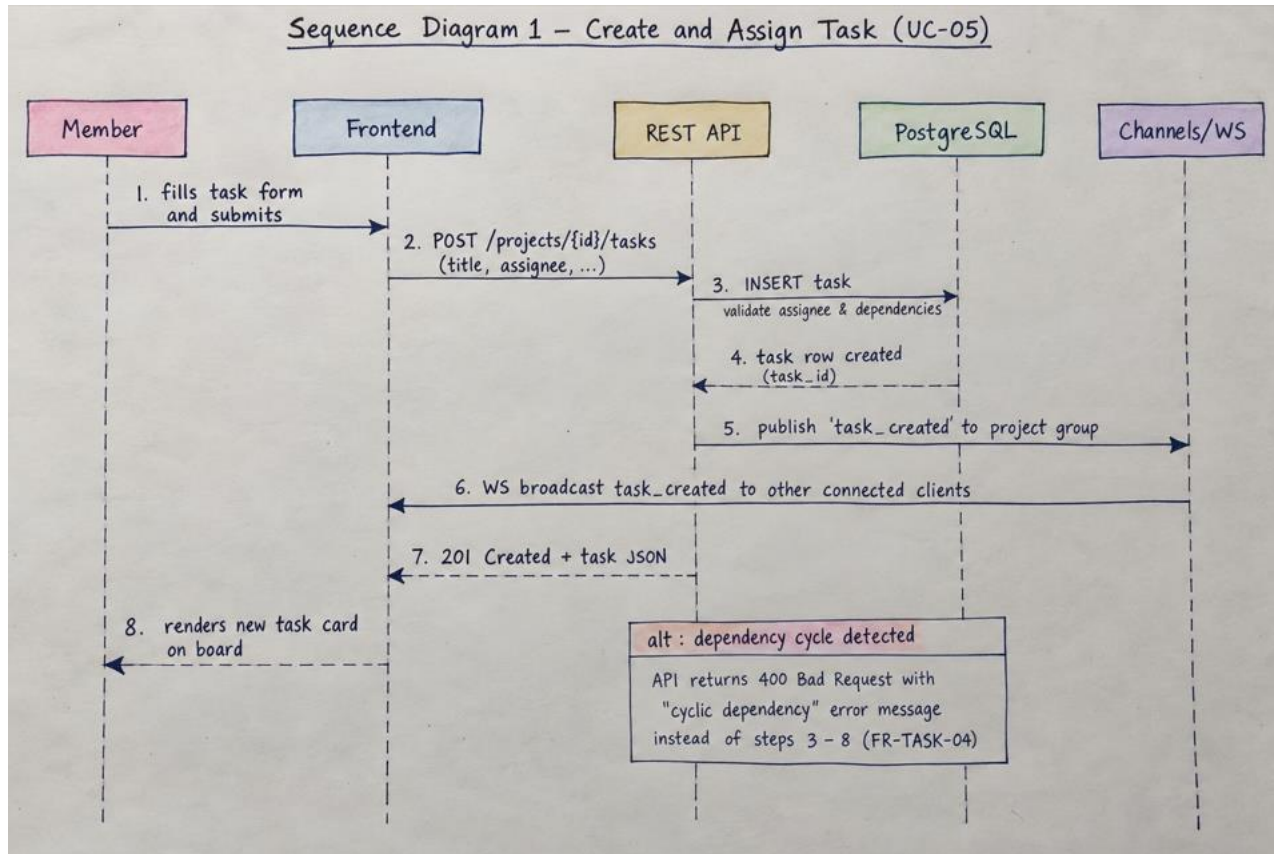


Figure 2: Sequence diagram - Create and Assign Task (UC-05, FR-TASK-02, FR-TASK-04)

This flow shows the dependency-cycle check happening inside the Project & Task Management module before the row is committed, so an invalid dependency never reaches the database and never triggers a spurious real-time broadcast.

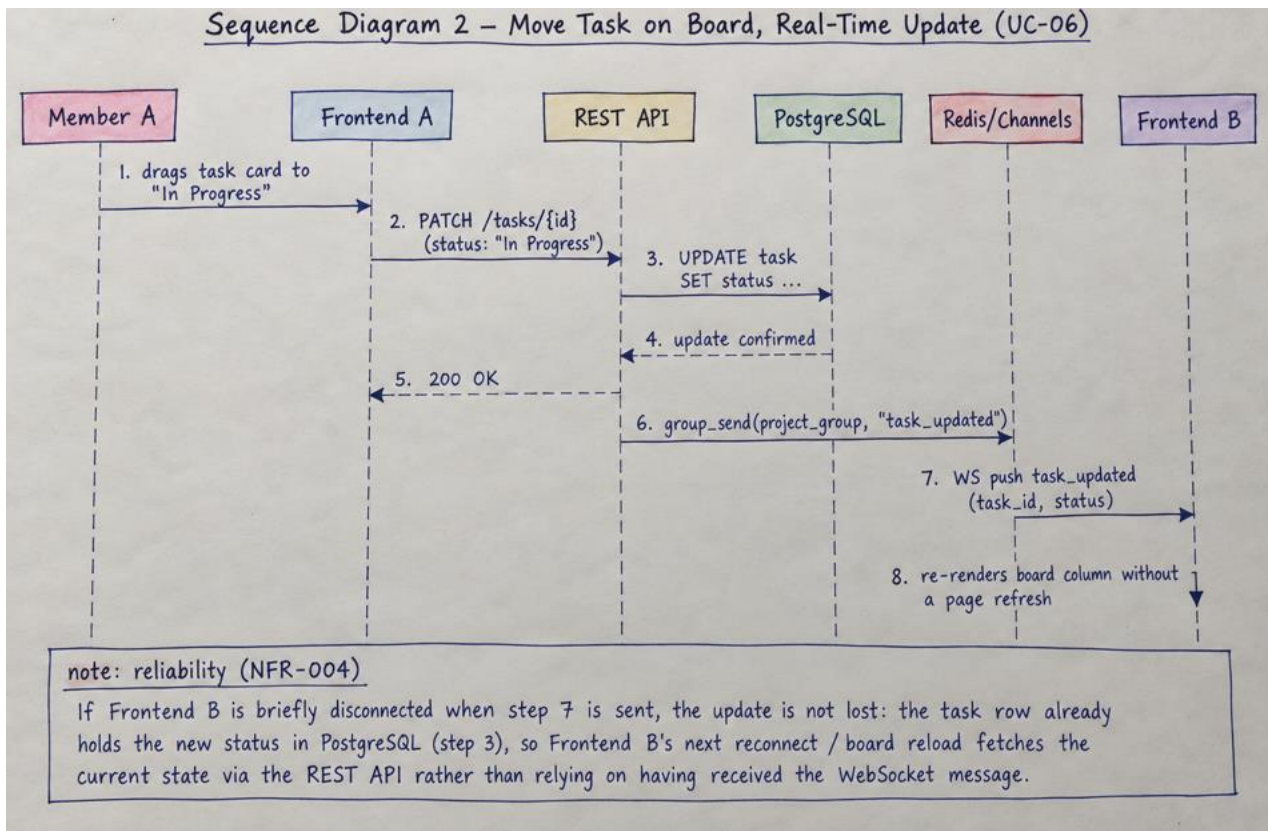


Figure 3: Sequence diagram - Move Task on Board with real-time broadcast (UC-06, FR-TASK-03, FR-RTC-01)

Here PostgreSQL is updated and acknowledges the write before the event is published to Redis, so the database is always the source of truth: a client that missed the WebSocket message (for example, because it was briefly disconnected) still sees the correct status on its next fetch, which is how NFR-004 is satisfied by design rather than by retry logic on the WebSocket path.

3.2 API Design

The REST API is versioned under `/api/v1/` and returns JSON for every response. Representative endpoints:

Method & Path	Description	Auth / Role
POST <code>/auth/register</code>	Create a new user account	None
POST <code>/auth/login</code>	Authenticate and receive a token	None

POST /workspaces	Create a workspace	Any authenticated user
POST /workspaces/{id}/invites	Generate a member invite link	Owner / Admin
PATCH /workspaces/{id}/members/{uid}	Change a member's role	Owner / Admin
POST /workspaces/{id}/projects	Create a project	Member+
POST /projects/{id}/tasks	Create and assign a task	Member+
PATCH /tasks/{id}	Update task fields, including status	Member+ (assignee or Owner for time)
POST /tasks/{id}/dependencies	Add a dependency between two tasks	Member+
POST /projects/{id}/backlog-items	Add a backlog item with story points	Member+
POST /sprints	Create a sprint and attach backlog items	Member+
POST /tasks/{id}/time-logs	Log time against a task	Assignee or Owner
POST /tasks/{id}/comments	Add a comment, optionally mentioning a user	Member+
GET /projects/{id}/analytics	Fetch burndown, velocity and workload metrics	Owner / Admin / Member
WS /ws/projects/{id}/	Subscribe to task_updated, task_created, comment_added, notification events	Valid token, project member

Every WebSocket message shares one envelope shape, {type, payload, timestamp}, where type is one of task_created, task_updated, comment_added or notification, so the frontend can dispatch on a single field regardless of which module produced the event.

3.3 Error Handling

All error responses share one envelope so the frontend has a single place to render errors: { "error": { "code": string, "message": string, "details": object | null } }.

HTTP Status	Used When	Example error.code
400 Bad Request	Input fails validation, or a dependency cycle is detected	cyclic_dependency, invalid_field
401 Unauthorized	Token missing, malformed or expired (SEC-004)	token_expired
403 Forbidden	Role does not permit the action (SEC-002)	insufficient_role
404 Not Found	Workspace, project or task does not exist or is soft-deleted	not_found
409 Conflict	Sprint start requested with an empty backlog, or duplicate email on register	empty_sprint, duplicate_email
500 Internal Server Error	Unhandled server-side failure; logged, never leaks internals to the client	internal_error

Unhandled exceptions are caught by a single DRF exception handler so that NFR robustness (Section 6.4) is met uniformly: the client always receives the error envelope above and a clear message instead of a raw stack trace, and the incident is logged server-side (SEC-006).

3.4 Other Design Considerations

3.4.1 Data Model Notes

The relational schema follows the ER diagram and field layout already published in SRS Section 4 and Appendix B. Two design additions sit on top of that schema: a soft-delete boolean on Workspace, Project and Task (SRS Safety Requirements, Section 6.2) so a delete is reversible until the final purge job runs, and a composite index on (project_id, status) for Task, since the Kanban board's primary query filters on exactly those two columns and must stay within the 2-second budget in NFR-001.

3.4.2 Consistency of Real-Time Updates

Because PostgreSQL is always written before an event is published (Section 3.1), the WebSocket channel only ever announces state that has already been durably committed, avoiding a class of bugs where connected clients could see a status that a slower REST client would then fail to confirm.

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering
Jackfruit Mini Project - Software Engineering (UE24CS341A)

Software Test Plan

for Project Management System

Field	Details
Test Plan Identifier	PMS-TP-1.0
Project Title	Project Management System
Group Number	Team 14
Version	1.0 (initial)
Prepared By	Samarth M H, Sameeksha Prasanna Kumar, Sanapureddy Hemanth Kumar Reddy, Sanjana Guttapally
Organization	PES University, Department of Computer Science and Engineering
Date Created	26 September 2026
Format Basis	IEEE Std 829-1998, Standard for Software Test Documentation
Related Documents	SRS v1.0; Software Architecture & Design Specification v1.0

Table of Contents

Table of Contents.....	2
1. Introduction	3
1.1 Purpose.....	3
1.2 Scope	3
1.3 References	3
1.4 Test Levels Overview.....	3
2. Test Items	4
3. Features to be Tested	4
3.1 In Scope	4
3.2 Out of Scope for This Cycle	5
4. Item Pass/Fail Criteria	5
5. Approach	5
5.1 Security Validation	5
6. Test Deliverables.....	6
7. Environmental Needs.....	6
8. Responsibilities	6
9. Schedule	7
10. Risks and Contingencies	7
11. Test Cases	8
12. Requirement Traceability Matrix (Test View)	10
13. Approvals.....	10

1. Introduction

1.1 Purpose

This Test Plan defines the scope, approach, resources and schedule of the testing activities for the Project Management System. It follows the IEEE 829 test plan outline and is intended for the development team, for the course instructor evaluating test coverage, and for anyone who needs to understand how the requirements in the SRS will be verified.

1.2 Scope

This plan covers functional testing of every feature group in SRS Section 5 (Authentication & Access Control, Project & Task Management, Agile Sprint & Time Management, and Real-Time Collaboration, Notifications & Analytics), the non-functional requirements in SRS Section 6.1, the security requirements in SRS Section 6.3, and the test-case updates requested for this submission.

1.3 References

- Software Requirements Specification (SRS) for Project Management System, Team 14, v1.0
- Software Architecture & Design Specification for Project Management System, v1.0
- IEEE Std 829-1998, Standard for Software Test Documentation

1.4 Test Levels Overview

Level	Focus	Primary Tooling
Unit	Individual functions/methods within each backend module and React component	Pytest, Jest
Integration	Module-to-module and WebSocket integration (e.g. task update -> broadcast)	Pytest + Django test client, Channels testing utilities
System	End-to-end use cases (UC-01 to UC-10) against a running instance	Manual + Postman/API test scripts
Acceptance	Course evaluation against SRS and this Test Plan	Instructor demo, this document

2. Test Items

The following items, each corresponding to a module described in the Architecture & Design Specification, are under test in this cycle:

Test Item	SRS Section	Owner
Authentication, Workspace & Access Control module	5.1	Samarth M H
Project & Task Management module (incl. Kanban board, dependency cycle detection)	5.2	Sameeksha Prasanna Kumar
Agile Sprint & Time Management module	5.3	Sanjana Guttapally
Real-Time Collaboration, Notifications & Analytics module	5.4	Sanapureddy Hemanth Kumar Reddy
Non-functional characteristics: performance, availability, reliability, scalability	6.1	All
Security controls (SEC-001 to SEC-006)	6.3	All

3. Features to be Tested

3.1 In Scope

- Registration, login, workspace creation, invites and RBAC (FR-AUTH-01 - FR-AUTH-04)
- Project/task creation, subtasks, Kanban board moves, dependency cycle detection (FR-TASK-01 - FR-TASK-04)
- Backlog management, sprint planning, time logging, burndown/velocity charts (FR-SPR-01 - FR-SPR-04)
- Real-time task board updates, comments/mentions, activity feed, notifications, analytics dashboard (FR-RTC-01 - FR-RTC-04)
- Performance, availability, reliability and scalability targets (NFR-001 - NFR-006)

- All security requirements (SEC-001 - SEC-006)

3.2 Out of Scope for This Cycle

- Internal correctness of third-party infrastructure: PostgreSQL's own storage engine, Redis's own pub/sub implementation, and browser rendering engines
- Load testing beyond the 50-concurrent-connection target defined in NFR-006
- Penetration testing beyond the basic checks listed in Section 5.1 of this plan

4. Item Pass/Fail Criteria

A test item passes when every test case mapped to it in Section 15 produces its expected result with no severity-1 (blocking) or severity-2 (major) defect open against it. A feature fails acceptance if any functional requirement's test case fails, or if a security or performance test case fails against the thresholds stated in SRS Section 6.

5. Approach

Testing follows the same black-box, requirement-driven approach used for validation planning in the SRS (Section 8): each functional requirement is exercised through the public REST/WebSocket API or the UI exactly as an end user would use it, rather than by inspecting internal code paths directly. Automated unit tests (Pytest for the backend, Jest for the frontend) run on every push through the Jenkins pipeline defined in SRS Section 10; integration and system tests are run before each sprint review; authorization tests are run for every role against every protected endpoint.

5.1 Security Validation

Security validation is treated as its own track within the approach, directly covering SRS Section 6.3:

Security Requirement	Validation Technique
SEC-001 - passwords never stored in plaintext	Code/configuration review of the password hasher; direct inspection of the users table to confirm only hashes are present
SEC-002 - Viewer cannot create/edit tasks	Authorization test: authenticate as a Viewer, attempt POST/PATCH/DELETE on a task, assert 403 Forbidden
SEC-003 - input is validated and sanitized	Negative/security tests: malformed payloads, oversized fields, and injection-style strings submitted

	to each write endpoint, asserting they are rejected with 400 and never reach the ORM unescaped
SEC-004 - expired/invalid tokens rejected	Security test: call a protected endpoint and open a WebSocket connection with an expired and with a malformed token, assert 401 / connection refusal
SEC-005 - removed member loses access immediately	Authorization test: remove a member mid-session, immediately retry their previously valid token against a protected endpoint, assert 403
SEC-006 - security events are logged	System test: trigger a failed login and a permission-denied event, confirm both appear in the security log

6. Test Deliverables

- This Test Plan document
- Automated unit test suites (Pytest, Jest) checked into the repository
- Test execution results / coverage reports from each CI run
- Completed test case log (Section 15) with pass/fail status per case
- Defect log for any failed test case

7. Environmental Needs

Category	Requirement
Hardware/hosting	Docker containers for backend, frontend, PostgreSQL and Redis, runnable on a course-provided or team laptop-class machine
Software	Python 3 + Django REST Framework + Django Channels, Node.js + React, PostgreSQL, Redis
Test tooling	Pytest, Jest, Postman or an equivalent API test client, a basic load-testing tool (e.g. Locust) for NFR-001/NFR-006
Browsers	Latest Chrome, Firefox and Edge on desktop, per SRS Section 2.4
CI/CD	Jenkins pipeline for automated test execution on push

8. Responsibilities

Team Member	Testing Responsibility
-------------	------------------------

Samarth M H	Authentication, workspace and access-control tests; token/security tests (SEC-001, SEC-004, SEC-005)
Sameeksha Prasanna Kumar	Project/task management tests, Kanban board tests, dependency-cycle negative tests
Sanjana Guttapally	Backlog, sprint planning and time-logging tests; burndown/velocity chart verification
Sanapureddy Hemanth Kumar Reddy	Real-time WebSocket tests, comments/mentions/notifications, analytics dashboard tests
All	Joint responsibility for performance/reliability tests (Section 6.1) and the remaining security tests (SEC-002, SEC-003, SEC-006)

9. Schedule

Testing activities map to Work Package WP7 ("Testing, CI/CD and security validation", 6 person-days, all members) in the SRS Section 9 project plan, running through Sprints 5-6 alongside WP8 (final validation and documentation), so that all test cases in Section 15 are executed and closed before the final report is prepared.

10. Risks and Contingencies

Risk	Contingency
WebSocket integration tests are flaky in CI due to timing	Add explicit wait/poll assertions with a generous timeout rather than fixed sleeps; rerun once automatically before marking a case failed
Limited time to run true load tests before the deadline	Scope NFR-001/NFR-006 verification to a smaller, representative concurrent-user count and document the extrapolation
A team member's module is not feature-complete before its test window	Reprioritise: write and review the test cases against the design now, execute them as soon as the feature lands, tracked against WP7's slack

11. Test Cases

The following ten test cases cover both functional and non-functional requirements, satisfying the minimum of seven to ten cases requested for this submission. Each case references the requirement it verifies.

ID	Objective	Related Req.	Preconditions	Key Steps	Expected Result	Type
TC-01	Register with a unique email succeeds	FR-AUTH-01	Application available; email not already registered	1. Submit registration form with valid, unused email and password	Account is created; success response returned	Functional
TC-02	Login returns an auth token	FR-AUTH-01	A registered account exists	1. Submit valid credentials to the login endpoint	200 OK with a valid authentication token	Functional
TC-03	Viewer cannot create a task	FR-AUTH-04 / SEC-002	Logged in as a Viewer on an existing project	1. Attempt POST /projects/{id}/tasks as Viewer	403 Forbidden; no task row created	Authorization
TC-04	Create and assign a task with subtasks	FR-TASK-02	Logged in as a Member; project exists	1. Create a task with two subtasks and an assignee	Task and subtasks are created; assignee is set correctly	Functional
TC-05	Task move updates board and broadcasts live	FR-TASK-03 / FR-RTC-01	Two clients connected to the same project's board	1. Client A drags a task to a new column 2. Observe Client B	Client A sees the new column; Client B receives the update via WebSocket without a manual refresh	Functional/Int
TC-06	Cyclic task dependency is rejected	FR-TASK-04	Two tasks A and B exist; A depends on B	1. Attempt to make B depend on A	400 Bad Request with a cyclic-dependency	Negative

					error; no dependency saved	
TC-07	Backlog item moves into a sprint with points	FR-SPR-01 / FR-SPR-02	Backlog has at least one item	1. Create a sprint 2. Move a backlog item with story points into it	Sprint is created; item is associated with the sprint and its points count toward the sprint total	Functional
TC-08	Comment with mention triggers a notification	FR-RTC-02 / FR-RTC-03	Task exists; two members are on the project	1. Member A comments on a task and mentions Member B	Comment is saved and appears in the activity feed; Member B receives an in-app notification	Functional
TC-09	Task board responds within budget under load	NFR-001 / NFR-006	50 simulated concurrent users connected to one project	1. Issue concurrent GET requests to the task board endpoint	95% of requests complete within 2 seconds; no dropped WebSocket connections	Non-functional (performance)
TC-10	No task data lost on WebSocket disconnect	NFR-004	A client is connected and a task update is in flight	1. Disconnect the client mid-update 2. Reconnect and reload the board	The task's latest status (already committed to PostgreSQL) is shown correctly after reconnect	Non-functional (reliability)

12. Requirement Traceability Matrix (Test View)

This matrix complements the SRS's own Appendix C by linking each requirement to the test case(s) in Section 11 that verify it, closing the loop between the SRS, the Architecture & Design Specification and this Test Plan.

Requirement ID	SRS Source	Verified By
FR-AUTH-01	SRS 5.1.3	TC-01, TC-02
FR-AUTH-02	SRS 5.1.3	System test - workspace creation walkthrough
FR-AUTH-03	SRS 5.1.3	System test - invite-link generation walkthrough
FR-AUTH-04	SRS 5.1.3	TC-03
FR-TASK-01	SRS 5.2.3	System test - project creation walkthrough
FR-TASK-02	SRS 5.2.3	TC-04
FR-TASK-03	SRS 5.2.3	TC-05
FR-TASK-04	SRS 5.2.3	TC-06
FR-SPR-01	SRS 5.3.3	TC-07
FR-SPR-02	SRS 5.3.3	TC-07
FR-SPR-03	SRS 5.3.3	System test - time-logging walkthrough
FR-SPR-04	SRS 5.3.3	System test - burndown/velocity chart check
FR-RTC-01	SRS 5.4.3	TC-05
FR-RTC-02	SRS 5.4.3	TC-08
FR-RTC-03	SRS 5.4.3	TC-08
FR-RTC-04	SRS 5.4.3	System test - analytics dashboard walkthrough
NFR-001	SRS 6.1	TC-09
NFR-004	SRS 6.1	TC-10
NFR-006	SRS 6.1	TC-09
SEC-001 - SEC-006	SRS 6.3	Section 5.1 of this plan (security validation track)

13. Approvals

Name	Role	Signature	Date
Samarth M H	Team Member		

Sameeksha Prasanna Kumar	Team Member		
Sanapureddy Hemanth Kumar Reddy	Team Member		
Sanjana Guttapally	Team Member		
Course Instructor	Approver		